Fresher Project, Part 1: Trajectory Simulation

Imperial College Rocketry — Flight Dynamics & Control

1 Overview

Over the next two weeks you will build a 3DOF trajectory simulator for a rocket and then use it to design a controller. This sheet covers **Part 1: the simulator**. You will be given a rocket and its launch conditions, and you will predict its trajectory from ignition on the pad up to apogee.

Part 2 builds directly on your Part 1 code, so write it to be extended. Keep the physics modular. In particular, **the drag calculation should be self-contained**, so that extra drag can easily be added later.

You may use any language but I would prefer you to stick with MATLAB, python or C/C++. Write the integration loop or the call to the solver yourself, do not use a rocket simulation library.

By the end of this week you should be able to:

- identify and model the forces acting on a rocket in flight to setup an initial value problem (IVP).
- implement a numerical integrator and use it to solve the IVP.

2 Conventions

Everyone must use these conventions.

Item	Convention
Reference frame	Right-handed, ground-fixed: x east, y north, z up. Origin at the base of the launch rail.
State	Position $\mathbf{r} = (x, y, z)$, velocity $\mathbf{v} = (v_x, v_y, v_z)$ relative to the ground, and mass m. So your state vector is $[x, y, z, v_x, v_y, v_z, m]^T$.
Units	SI throughout. Angles in the input files are in degrees.
Altitude	z is height above ground level. The atmosphere model needs altitude above sea level: $h = z + h_{\text{site}}$.
Time	$t = 0$ at ignition, i.e. the first sample of the thrust curve.
Rail angles	Elevation θ_{rail} measured up from horizontal (90° is vertical). Heading ψ_{rail} measured clockwise from north.
Wind	You can exclude wind for this project.

3 Data Provided

You will be given a parameter script and two data files.

Parameter	Symbol	Units	Source
Dry mass	m_{dry}	kg	parameter script
Propellant mass	m_{prop}	kg	parameter script
Reference diameter	d	m	parameter script
Rail length	L_{rail}	m	parameter script
Rail elevation	θ_{rail}	deg	parameter script
Rail heading	ψ_{rail}	deg	parameter script
Launch site altitude	h_{site}	m ASL	Assume 0
Thrust curve	$T(t)$	N vs s	Thrust curve csv
Drag coefficient	$C_D(M)$	– vs Mach	Cd csv

The drag coefficient uses the reference area $A_{\text{ref}} = \pi d^2/4$.

4 Modelling Assumptions

Use these assumptions. If you make any others, state them in your write-up.

1. **Point mass.** The rocket has three translational degrees of freedom. There are no rotational dynamics.
2. **Zero angle of attack.** Once off the rail, the rocket always points along its velocity *relative to the air*. Thrust and drag act along this direction.
3. **Launch rail.** While on the rail, the rocket moves only along the rail direction and the rail is frictionless. The rocket leaves the rail once it has travelled L_{rail} along it.
4. **Earth.** Flat and non-rotating, with constant $g = 9.81$ m/s^2.
5. **Mass.** Propellant mass decreases in proportion to the total impulse delivered so far. All of the propellant has been used by the end of the thrust curve.
6. **Atmosphere.** International Standard Atmosphere (ISA).
7. **End condition.** The simulation ends at apogee. You do not need to model the descent.

5 Things You Will Need to Research

The equations are deliberately not given here. Everything below is standard and well documented. Part of the project is finding it, understanding it and being able to justify your choices. If you have been stuck on something for a while we are heppy to help.

- **Equations of motion for a body whose mass changes.** Why is writing $\mathbf{F} = \frac{d}{dt}(m\mathbf{v})$ a trap?
- **Aerodynamic drag.** The drag equation, dynamic pressure and reference area (if you're aero you should have learnt these already).
- **The International Standard Atmosphere.** Temperature, pressure, density and speed of sound as functions of altitude. Check whether your apogee goes above the troposphere.
- **Mach number.** What it is, and why C_D changes so much between Mach 0.8 and 1.2.
- **Interpolating tabulated data.** Linear interpolation is fine. Think about what happens outside the range of the table.

- **Numerical integration.** Explicit Euler, fourth-order Runge–Kutta (RK4) and adaptive-step solvers (`scipy.integrate.solve_ivp`, `ode45`). Many of you wont have encountered these before so definitely worth reading into.
- **Geometry.** Turning a rail elevation and heading into a unit vector in the frame above.

6 Required Outputs

Scalars: report these for each wind case.

Quantity	Units
Apogee (AGL)	m
Time to apogee	s
Maximum speed	m/s
Maximum Mach number	–
Rail exit speed	m/s
Burnout time and altitude	s, m
Horizontal distance from the pad at apogee	m

Plots: produce these for your simulated trajectory.

- altitude against time;
- speed and Mach number against time;
- mass against time (a sanity check);
- ground track (y against x);
- altitude against horizontal distance from the pad.

7 Extensions (if you finish early)

- Add wind effects and make them scale with altitude.
- Run a Monte Carlo simulation over uncertainty in mass, C_D and wind, and plot the spread of apogee.
- Model the descent under a parachute and predict where the rocket lands.